

Deadline-Aware Online Optimization for Mobile Edge Computing via Age-Layered Queues

Xin Gao*, Hongwei Shi[†], Chen Su*, Zhi Zheng* and Tao Huang^{‡*}

*Future Network Center, Purple Mountain Laboratories, Nanjing 211111, China

[†]School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China

[‡]State Key Laboratory of Networking and Switching Technology,

Beijing University of Posts and Telecommunications, Beijing 100876, China

Emails: {gaoxin, shihongwei, suchen, zhengzhi, huangt}@pmlabs.com.cn

Abstract—Mobile edge computing (MEC) faces challenges in meeting the stringent, deterministic delay requirements of mission-critical 6G services due to stochastic dynamics and coupled resources. This paper investigates online resource allocation for MEC networks with per-workload end-to-end deadlines. We develop an age-layered queue model with age-priority service to capture and process workloads based on urgency. We then formulate a long-term stochastic optimization problem to minimize energy consumption and deadline-violation penalties. To solve it, we propose the Age-layered Deadline-aware Dynamic Optimization (ADDO) algorithm. Based on Lyapunov drift-plus-penalty theory, ADDO jointly optimizes local computing, offloading, and edge CPU allocation without requiring prior traffic or channel statistics. Theoretical analysis establishes an $O(1/V)$ cost-optimality gap and an upper bound on the average violated workload. Simulations demonstrate that ADDO consistently achieves the lowest deadline-violation ratio and attains near-zero violations across a broad range of evaluated settings.

Index Terms—edge computing, deadline-aware scheduling, age-layered queues, Lyapunov optimization, resource allocation.

I. INTRODUCTION

With the rapid development of artificial intelligence, the Internet of Things (IoT), and next-generation mobile communications, future 6G networks are expected to support a wide range of mission-critical services, such as autonomous driving, industrial automation, and immersive interaction [1]. These applications require not only high data rates and ubiquitous connectivity, but also stringent and verifiable end-to-end delay guarantees. As a result, deterministic latency provisioning has emerged as a fundamental requirement for next-generation wireless architectures.

Mobile edge computing (MEC) is widely recognized as a pivotal paradigm for enabling such latency-sensitive services. By migrating computation and storage resources from centralized clouds to edge nodes in close proximity to end users [2], MEC can significantly reduce service latency, alleviate backbone traffic, and enhance system responsiveness. However, to fully support mission-critical applications, MEC systems must guarantee that each workload is completed within a prescribed deadline, making deterministic delay provisioning a central design objective.

Despite its importance, existing studies only partially address this challenge. On the communication side, deterministic

networking technologies such as Time-Sensitive Networking (TSN) and Deterministic Networking (DetNet) mainly focus on predictable transmission latency, while often overlooking the communication–computation coupling inherent in MEC systems [3]. On the MEC side, a substantial body of work on resource allocation targets average-delay minimization, probabilistic delay guarantees, or robust design under uncertainty [4]–[6]. Although these studies provide useful insights, they do not explicitly enforce hard end-to-end deadlines in stochastic MEC environments. In addition, while Lyapunov-based online optimization has been widely adopted for dynamic MEC control due to its independence from prior statistical knowledge [7], [8], existing frameworks are primarily developed for time-average performance metrics and are not directly tailored to deterministic deadline enforcement.

To bridge this gap, this paper investigates online resource allocation in MEC networks subject to hard end-to-end delay constraints. The key challenge lies in how to explicitly represent workload urgency in a stochastic computation–communication coupled system, while preserving the online and distribution-free advantages of Lyapunov optimization. To this end, we develop an age-layered queueing model that partitions both local and edge backlogs according to workload age. Under this model, any workload that reaches its maximum admissible age without being completed is explicitly identified as overdue. We further introduce an age-priority service discipline, under which older workloads are always served first, so that the resulting control policy is inherently deadline-aware.

Based on the above model, we formulate a long-term stochastic optimization problem to minimize the average system cost, including local computing energy, transmission energy, edge computing energy, and deadline-violation penalties, subject to queue stability and resource constraints. We then propose an age-layered deterministic-delay optimization (ADDO) algorithm based on Lyapunov drift-plus-penalty optimization. The proposed algorithm jointly determines local computing, task offloading, and edge CPU allocation in an online manner without requiring prior knowledge of traffic arrival or channel-state statistics.

Compared with existing studies, the proposed framework has three main distinguishing features. First, instead of focusing on average delay, probabilistic reliability, or virtual-

queue-based surrogate delay metrics, it directly targets hard end-to-end delay constraints at the workload level. Second, by introducing an age-layered queueing model together with an age-priority service discipline, the proposed method makes deadline urgency explicitly visible to the controller in both local and edge queues. This allows the controller to identify critical workloads that are close to deadline violation and to intervene earlier through local computing, offloading, and edge CPU allocation, thereby enabling more effective deadline protection than indirect average-based regulation. Third, the resulting algorithm operates online without requiring prior statistical knowledge, while still admitting provable performance guarantees on the cost–backlog tradeoff. In this way, the proposed framework bridges deterministic delay control and stochastic online optimization in computation–communication coupled MEC systems.

The main contributions of this paper are summarized as follows:

- We develop an age-layered queueing framework for MEC systems with hard end-to-end delay constraints. By explicitly tracking workload age in both local and edge queues and combining it with an age-priority service discipline, the proposed model establishes a tractable connection between deterministic delay requirements and queue evolution.
- We formulate a long-term stochastic optimization problem that jointly captures communication cost, computation cost, and deadline-violation penalties under stochastic arrivals and time-varying wireless channels. This formulation explicitly characterizes the tradeoff between energy expenditure and deadline protection.
- We propose the ADDO algorithm based on Lyapunov drift-plus-penalty optimization. The algorithm makes per-slot decisions on local computing, task offloading, and edge CPU allocation in an online and distribution-free manner, with closed-form local control and efficient bisection-based edge resource allocation.
- We establish theoretical performance guarantees for the proposed algorithm, showing an $O(1/V)$ cost-optimality gap and an $O(V)$ average-backlog bound further capped by a finite age-induced upper bound, where $V > 0$ is the Lyapunov control parameter. Numerical results further show that ADDO achieves the lowest deadline-violation ratio among benchmark policies and can completely eliminate deadline violations in certain scenarios.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system model. Section IV formulates the long-term optimization problem. Section V develops the proposed ADDO algorithm and its theoretical analysis. The numerical results are presented in Section VI. Finally, Section VII concludes the paper.

II. RELATED WORK

A. Deterministic Delay Guarantees

Deterministic communication has been widely investigated for latency-critical services. In wired networks, Time-Sensitive

Networking (TSN) provides bounded-latency communication through time synchronization and gate control scheduling [9], while Deterministic Networking (DetNet) extends deterministic forwarding and resource reservation mechanisms to IP networks [10]. In wireless systems, ultra-reliable low-latency communication (URLLC) has motivated extensive studies on delay–reliability tradeoffs based on finite blocklength coding, effective capacity, and stochastic network calculus [11], [12].

These works provide important foundations for predictable transmission latency. However, they mainly focus on the communication layer and do not explicitly address end-to-end deterministic latency in MEC systems, where wireless transmission, task offloading, queueing, and edge execution are tightly coupled.

B. Latency Optimization and Online Control in MEC

Latency optimization has been a major topic in MEC research. Existing studies have investigated joint task offloading and resource allocation to reduce delay and improve energy–delay tradeoffs. Representative approaches include risk-aware formulations based on chance constraints or CVaR [?], [13], queueing-theoretic delay analysis [?], and robust optimization under uncertainty [5]. These methods provide useful tools for latency-aware MEC design, but they mainly target average delay performance, probabilistic guarantees, or robustness under prescribed uncertainty models, rather than hard per-task deadline enforcement.

To address time-varying arrivals, channel conditions, and computing demands, prior work has also applied stochastic optimization, Markov decision processes, reinforcement learning, and Lyapunov-based stochastic network optimization to dynamic MEC resource management [14]–[18]. Among them, Lyapunov-based methods are particularly attractive because they enable online control without requiring prior statistical knowledge. Nevertheless, existing online MEC frameworks still mainly optimize time-average metrics such as average delay or long-term cost. Even when virtual queues are introduced, the resulting guarantees are typically expressed in terms of average violations or probabilistic performance, rather than deterministic end-to-end delay guarantees for individual tasks.

In summary, existing studies on deterministic communication mainly address predictable transmission latency, while most MEC resource-allocation methods focus on average-delay optimization, probabilistic delay guarantees, robust design under uncertainty, or distributed coordination with delay guarantees. In particular, URLLC-oriented MEC schemes typically characterize latency reliability through probabilistic queue-length constraints or extreme-event statistics, whereas Lyapunov-based MEC approaches usually enforce deadline-related requirements through virtual queues and optimize long-term average performance. Different from these lines of work, our method makes deadline urgency directly observable to the controller by tracking workload age in both local and edge queues and prioritizing older workloads in service. This provides a more fine-grained view of deadline risk than conventional backlog-based or virtual-queue-based formula-

tions, which mainly regulate delay indirectly through average quantities. As a result, the controller can distinguish between workloads that are merely accumulated and those that are close to deadline violation, and can therefore intervene earlier through local computing, offloading, and edge CPU allocation. Such a mechanism is particularly advantageous under hard delay constraints, since it enables more timely protection of critical workloads and leads to lower deadline-violation ratios in stochastic MEC systems, while still preserving the distribution-free and provably efficient nature of Lyapunov-based online optimization.

III. SYSTEM MODEL

We consider a slotted dynamic mobile edge computing (MEC) network consisting of one edge server¹ and a set of users $\mathcal{K} \triangleq \{1, \dots, K\}$. Time is divided into equal-length slots indexed by $t \in \{0, 1, 2, \dots\}$, where each slot has duration δ . The edge server is co-located with an access point, and all users can either process their workloads locally or offload them to the edge server for remote execution.

A. Workload and Resource Model

For each user $i \in \mathcal{K}$, let $A_i(t)$ denote the workload arriving at the beginning of time slot t , measured in bits.² The arrival process $\{A_i(t)\}$ is stochastic and time-varying, and is assumed to be bounded as $0 \leq A_i(t) \leq A_{i,\max}$, $\forall t$. All newly arrived workload is first buffered and then scheduled for processing either locally or at the edge server.

Each workload generated by user i is associated with a *hard end-to-end latency deadline* $D_i \in \mathbb{Z}_+$, meaning that every workload unit must be completed within D_i time slots after its arrival.³

Each workload bit of user i requires c_i CPU cycles for execution. User i is equipped with a local processor, whose CPU frequency in slot t is denoted by $f_i^L(t)$. Meanwhile, the edge server allocates a CPU frequency $f_i^E(t)$ to process the offloaded workload of user i . Accordingly, the maximum amount of workload that can be processed within slot t (in bits) by local and edge computing, respectively, is given by

$$\mu_i^L(t) = \delta f_i^L(t)/c_i, \quad (1)$$

$$\mu_i^E(t) = \delta f_i^E(t)/c_i, \quad (2)$$

where δ denotes the duration of one time slot. These service rates characterize the maximum executable workload under the allocated CPU resources.

The local CPU frequency of each user is constrained by the hardware capability as

$$0 \leq f_i^L(t) \leq f_{i,\max}^L, \quad \forall i, t, \quad (3)$$

while the edge server has a total CPU frequency budget shared among all users, given by

$$\sum_{i \in \mathcal{K}} f_i^E(t) \leq f_{\max}^E, \quad f_i^E(t) \geq 0, \quad \forall i, t, \quad (4)$$

where $f_{i,\max}^L$ and $f_{\max}^E$ denote the maximum local CPU frequency and the total edge CPU-frequency budget, respectively.

Each user may offload part of its local workload over the wireless uplink. Let $p_i(t)$ denote the transmit power of user i in slot t , which subject to

$$0 \leq p_i(t) \leq p_{i,\max}, \quad \forall i, t. \quad (5)$$

Then the amount of workload that can be offloaded in slot t is modeled as

$$\mu_i^T(t) = \delta B_i \log_2(1 + p_i(t)H_i(t)/(N_0 B_i)), \quad (6)$$

where $H_i(t)$ denotes the corresponding channel power gain, B_i is the allocated bandwidth, and N_0 is the noise power spectral density. To capture both large-scale attenuation and small-scale fading, $H_i(t)$ is modeled as

$$H_i(t) = \beta_0 \left(\frac{d_0}{d_i(t)} \right)^\alpha \xi_i(t), \quad (7)$$

where $d_i(t)$ is the distance from user i to the edge node, $d_0 = 1$ m is the reference distance, β_0 denotes the channel power gain at the reference distance, and α is the path-loss exponent. The small-scale fading term is denoted by $\xi_i(t) = |g_i(t)|^2$, where $g_i(t) \sim \mathcal{CN}(0, 1)$. Hence, $\xi_i(t)$ follows an exponential distribution with unit mean. We assume a frequency-flat block fading channel, i.e., $H_i(t)$ remains constant within each slot and changes independently across different slots.

B. Age-Layered Queue Model

Each user maintains one *local queue* to buffer newly arrived workloads. At the edge side, the edge server maintains K *logical edge queues*, one for each user to preserve user-specific queue states and allow fine-grained edge CPU scheduling across users under a shared computation budget.

Let $Q_i^L(t)$ and $Q_i^E(t)$ denote the total local and edge backlog of user i at the beginning of slot t , respectively. Newly generated workload first enters $Q_i^L(t)$. From the local queue, workload can either be executed locally or offloaded to the corresponding logical edge queue. The workload offloaded in slot t is assumed to arrive at the edge server at the beginning of slot $t + 1$, which captures a one-slot transmission latency and is consistent with the slotted decision model. The edge server then processes the received workload and returns the result to the user. Since the size of the computation result is typically much smaller than the input data, the downlink feedback delay is neglected, which is standard in MEC modeling.

To explicitly model the deterministic end-to-end delay requirement, we adopt the age-layered queue representation with age index $a \in \{0, 1, \dots, D_i - 1\}$ for each user $i \in \mathcal{K}$, where D_i is the delay bound in slots. An age- a workload has stayed in the system for a slots. Hence, any workload in layer $a = D_i - 1$ that is not completed within the

¹The proposed framework can be extended to multi-server MEC systems by augmenting the offloading action space and server-side queue states.

²Equivalently, the workload can be measured in packets or tasks after proper normalization.

³To make the model meaningful, we assume $D_i \geq 2$ for each user $i \in \mathcal{K}$.

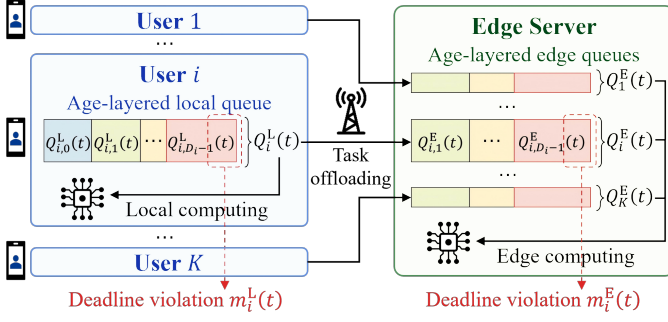


Fig. 1: Age-layered queueing architecture for deadline-aware MEC.

current slot will violate the deterministic delay bound and thus become overdue. Such overdue workload is either removed from the system or immediately processed through an urgent computation service.

For each user i , let $Q_{i,a}^L(t)$ and $Q_{i,a}^E(t)$ denote the local and edge workloads of age a at the beginning of slot t , respectively. The corresponding total local and edge backlogs are

$$Q_i^L(t) = \sum_{a=0}^{D_i-1} Q_{i,a}^L(t), \quad Q_i^E(t) = \sum_{a=0}^{D_i-1} Q_{i,a}^E(t). \quad (8)$$

Fig. 1 illustrates the proposed age-layered queueing architecture.

C. Age-priority Service Discipline

For age $a < D_i$, let $x_{i,a}(t)$ denote the amount of age- a workload locally processed by user i in slot t , and let $u_{i,a}(t)$ denote the amount of age- a workload offloaded in slot t . Similarly, let $y_{i,a}(t)$ denote the amount of age- a workload of user i processed at the edge in slot t . These *age-wise service variables* satisfy

$$x_{i,a}(t), u_{i,a}(t), y_{i,a}(t) \geq 0, \quad (9)$$

and

$$\sum_{a=0}^{D_i-1} x_{i,a}(t) \leq \mu_i^L(t), \quad (10)$$

$$\sum_{a=0}^{D_i-1} u_{i,a}(t) \leq \mu_i^T(t), \quad (11)$$

$$\sum_{a=0}^{D_i-1} y_{i,a}(t) \leq \mu_i^E(t), \quad (12)$$

where $\mu_i^L(t)$, $\mu_i^T(t)$, and $\mu_i^E(t)$ are the local computing, offload transmission, and edge computing capacities, respectively.

We impose an *age-priority service discipline* that always serves older workloads first. At the local side, in each slot, user i first allocates local computation to the largest-age layers and only then uses the residual transmission capacity to offload younger workloads. At the edge side, the server also processes larger-age workloads first. This rule is natural under hard deadline constraints and is without loss of modeling generality,

and it is optimal for minimizing recourse under deterministic deadlines, similar to earliest-deadline-first (EDF).

The the age-wise service variables $x_{i,a}(t)$, $u_{i,a}(t)$, and $y_{i,a}(t)$ are uniquely induced by the age-priority service discipline as follows.

1) *Local computing*: Specifically, for each user i , local computing is performed first in descending age order. For $a = D_i - 1$ we have

$$x_{i,D_i-1}(t) = \min \{Q_{i,D_i-1}^L(t), \mu_i^L(t)\}, \quad \forall i, t, \quad (13)$$

and for $a = D_i - 2, D_i - 3, \dots, 0$ we have

$$x_{i,a}(t) = \min \left\{ Q_{i,a}^L(t), \mu_i^L(t) - \sum_{\alpha=a+1}^{D_i-1} x_{i,\alpha}(t) \right\}, \quad \forall i, t \quad (14)$$

2) *Offloading*: After local computing decisions are completed, the remaining local backlog becomes

$$\hat{Q}_{i,a}^L(t) = Q_{i,a}^L(t) - x_{i,a}(t), \quad \forall a, i, t. \quad (15)$$

We prohibits offloading of age- $(D_i - 1)$ workload to prevent deadline violation during transmission. Therefore, we impose

$$u_{i,D_i-1}(t) = 0, \quad \forall i, t. \quad (16)$$

For the remaining age layers, offloading is also carried out in descending age order over the residual eligible local backlog. Specifically, for $a = D_i - 2$ we have

$$u_{i,D_i-2}(t) = \min \left\{ \hat{Q}_{i,D_i-2}^L(t), \mu_i^T(t) \right\}, \quad \forall i, t, \quad (17)$$

and for $a = D_i - 3, D_i - 4, \dots, 0$ we have

$$u_{i,a}(t) = \min \left\{ \hat{Q}_{i,a}^L(t), \mu_i^T(t) - \sum_{\alpha=a+1}^{D_i-2} u_{i,\alpha}(t) \right\}, \quad \forall i, t \quad (18)$$

3) *Edge computing*: At the edge side, the server also follows the oldest-first rule for each user's logical edge queue. Specifically, for $a = D_i - 1$ we have

$$y_{i,D_i-1}(t) = \min \{Q_{i,D_i-1}^E(t), \mu_i^E(t)\}, \quad \forall i, t, \quad (19)$$

and for $a = D_i - 2, D_i - 3, \dots, 0$ we have

$$y_{i,a}(t) = \min \left\{ Q_{i,a}^E(t), \mu_i^E(t) - \sum_{\alpha=a+1}^{D_i-1} y_{i,\alpha}(t) \right\}, \quad \forall i, t \quad (20)$$

Under the above construction, the age-wise service variables are uniquely determined by the aggregate capacities and queue states.

D. Deadline-Violation Modeling

For user i , define the local and edge deadline-violation amounts in slot t as

$$m_i^L(t) = Q_{i,D_i-1}^L(t) - x_{i,D_i-1}(t), \quad (21)$$

$$m_i^E(t) = Q_{i,D_i-1}^E(t) - y_{i,D_i-1}(t). \quad (22)$$

Note that under the age-priority service discipline, we have

$$m_i^L(t) = [Q_{i,D_i-1}^L(t) - \mu_i^L(t)]^+, \quad (23)$$

$$m_i^E(t) = [Q_{i,D_i-1}^E(t) - \mu_i^E(t)]^+, \quad (24)$$

where $[\cdot]^+ = \max\{\cdot, 0\}$. The quantity $m_i(t)$ measures the number of workload bits that miss the deterministic delay bound in slot t . Such overdue workload is removed from the system or forcibly completed through an urgent computation service. Each bit of deadline violation on the user device incurs a violation penalty of $\phi_i^L > 0$. Similarly, each bit of deadline violation on the edge server incurs a violation penalty of $\phi_i^E > 0$. The total deadline violation cost is $\phi_i^L m_i^L(t) + \phi_i^E m_i^E(t)$.

The quantity $m_i(t)$ denotes the number of workload bits that violate the deterministic delay bound in slot t . Such overdue workload is either removed from the system or processed through an urgent computation service. Each bit of deadline violation at the user device incurs a penalty $\phi_i^L > 0$, while each bit of deadline violation at the edge server incurs a penalty $\phi_i^E > 0$. Accordingly, the total deadline violation cost is given by

$$r(t) = \phi_i^L m_i^L(t) + \phi_i^E m_i^E(t) \quad (25)$$

E. Queue Evolution

1) *Local queue dynamics*: New arrivals enter the age-0 local queue at the beginning of each slot:

$$Q_{i,0}^L(t) = A_i(t), \quad \forall i, t. \quad (26)$$

During slot t , a portion of age- a workload is either processed locally, denoted by $x_{i,a}(t)$, or offloaded to the edge server, denoted by $u_{i,a}(t)$. The remaining workload ages by one slot. Therefore, for $a = 0, 1, \dots, D_i - 2$, the queue evolution is given by

$$Q_{i,a+1}^L(t+1) = Q_{i,a}^L(t) - x_{i,a}(t) - u_{i,a}(t). \quad (27)$$

By summing (26) and (27) over all age layers, and using (10), (11), and (21), the total local backlog evolves as

$$\begin{aligned} Q_i^L(t+1) &= Q_i^L(t) - m_i^L(t) + A_i(t+1) \\ &\quad - \sum_{a=0}^{D_i-1} x_{i,a}(t) - \sum_{a=0}^{D_i-1} u_{i,a}(t) \\ &\leq [Q_i^L(t) - m_i^L(t) - \mu_i^L(t) - \mu_i^T(t)]^+ \\ &\quad + A_i(t+1). \end{aligned} \quad (28)$$

2) *Edge queue dynamics*: Since no workload is directly generated at the edge, the age-0 edge layer is identically zero, i.e.,

$$Q_{i,0}^E(t) = 0, \quad \forall i, t. \quad (29)$$

The offloaded workload experiences one-slot transmission delay. Specifically, the amount $u_{i,a}(t)$ offloaded from the local age- a queue in slot t arrives at the edge server at the beginning of slot $t+1$ with age $a+1$. Meanwhile, edge computing processes $y_{i,a}(t)$ amount of workload from the age- a queue. Therefore, for $a = 0, 1, \dots, D_i - 2$, the edge queue evolves as

$$Q_{i,a+1}^E(t+1) = Q_{i,a}^E(t) - y_{i,a}(t) + u_{i,a}(t). \quad (30)$$

By summing (30) and (27) over all age layers, and using (11), (12), and (22), the total edge backlog evolves as Similarly, summing (30) over all age layers yields

$$\begin{aligned} Q_i^E(t+1) &= Q_i^E(t) - m_i^E(t) - \sum_{a=0}^{D_i-1} y_{i,a}(t) + \sum_{a=0}^{D_i-1} u_{i,a}(t) \\ &\leq [Q_i^E(t) - m_i^E(t) - \mu_i^E(t)]^+ + \mu_i^T(t). \end{aligned} \quad (31)$$

F. Feasible Control Policy

In each slot t , the system controller observes the network state

$$\Theta(t) \triangleq \{Q_{i,a}^L(t), Q_{i,a}^E(t), A_i(t), H_i(t), \forall i, a\}, \quad (32)$$

and makes an online decision on local CPU frequencies, transmit powers, edge CPU allocation, and age-dependent service/offloading variables, i.e.,

$$\chi(t) \triangleq \{f_i^L(t), p_i(t), f_i^E(t), \forall i, a\}. \quad (33)$$

The age-wise service variables $\{x_{i,a}(t), u_{i,a}(t), y_{i,a}(t), \forall i, a\}$ are implicitly determined by the age-priority service discipline.

The objective of online optimization is to dynamically adapt $\chi(t)$ to stochastic arrivals and time-varying channels while guaranteeing the deterministic delay requirement. Depending on the design target, the system may minimize a long-term cost such as weighted communication-computation energy consumption, edge resource usage, or a network-wide utility loss, while preserving queue feasibility and deadline compliance.

IV. PROBLEM FORMULATION

Under the system model developed in Sec. III, the objective is to design an online control policy that dynamically determines $\chi(t)$ in each slot to minimize the long-term system cost while guaranteeing deterministic delay.

A. Per-Slot Cost Function

In each slot t , the system incurs cost from three regular operations, namely local computing, wireless offloading, and edge computing, as well as the deadline violation cost. Accordingly, the per-slot system cost is defined as

$$\begin{aligned} C(t) &= \sum_{i \in \mathcal{K}} (C_i^L(t) + C_i^T(t) + C_i^E(t)) \\ &\quad + \sum_{i \in \mathcal{K}} (\phi_i^L m_i^L(t) + \phi_i^E m_i^E(t)), \end{aligned} \quad (34)$$

where $C_i^L(t)$, $C_i^T(t)$, $C_i^E(t)$ are local computing cost, transmission cost and edge computing cost, respectively.

1) *Local computing cost*: For user i , the local CPU frequency in slot t is denoted by $f_i^L(t)$. Following the standard DVFS model, the local computing energy consumption is proportional to the square of the CPU frequency per cycle, yielding the per-slot local computing cost

$$C_i^L(t) = \delta \kappa (f_i^L(t))^3, \quad (35)$$

where $\kappa > 0$ is the effective switched-capacitance coefficient.

2) *Transmission cost*: Let $p_i(t)$ denote the uplink transmit power of user i in slot t . The transmission cost is defined as the uplink transmission energy consumed during one slot:

$$C_i^T(t) = \delta p_i(t). \quad (36)$$

This expression explicitly captures the increasing energy cost required to support larger offloading volumes under a given channel state.

3) *Edge computing cost*: In slot t , the edge computing energy consumption for processing workload from user i is

$$C_i^E(t) = \delta \kappa (f_i^E(t))^3, \quad (37)$$

where $\kappa > 0$ is the effective switched-capacitance coefficient at the edge.

The long-term optimization aims to balance local energy consumption, transmission energy consumption, and deadline violation cost under stochastic arrivals and time-varying wireless channels. Therefore, the proposed model not only guarantees deterministic delay but also preserves a meaningful tradeoff between deadline reliability and operational cost.

B. Long-Term Optimization Problem

We define the long-term time-average expectation of system cost $\bar{C}$ and total queue backlog $\bar{Q}$ as follows:

$$\bar{C} = \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[C(t)], \quad (38)$$

$$\bar{Q} = \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i \in \mathcal{K}} (\mathbb{E}[Q_i^L(t)] + \mathbb{E}[Q_i^E(t)]). \quad (39)$$

Based on the per-slot cost defined in (34), we aim to design an online control policy that minimizes the long-term time-average system cost.

Specifically, let $\{\chi(t)\}_{t=0}^{\infty}$ denote a sequence of feasible control actions adapted to the observed network state $\Theta(t)$. The long-term optimization problem is formulated as

$$\begin{aligned} (\mathbf{P1}) \quad & \min_{\{\chi(t)\}} \bar{C} \\ \text{s.t.} \quad & \bar{Q} < \infty, \\ & (3), (4), (5), \\ & \text{Age-priority service discipline (Sec. III-C).} \end{aligned}$$

Here, the expectation is taken with respect to the randomness of arrivals and wireless channel states.

Due to the stochastic nature and the coupled dynamics, problem **(P1)** is difficult to solve directly. In the next section, we develop a Lyapunov optimization framework to derive an efficient online algorithm that makes per-slot decisions while provably minimizing the long-term cost.

V. ALGORITHM DESIGN

A. Lyapunov Framework

To solve problem **(P1)**, we define the quadratic Lyapunov function as

$$L(t) = \frac{1}{2} \sum_{i \in \mathcal{K}} \left[(Q_i^L(t))^2 + (Q_i^E(t))^2 \right]. \quad (40)$$

The one-slot conditional Lyapunov drift is

$$\Delta(t) = \mathbb{E}[L(t+1) - L(t) \mid \mathbf{Q}(t)], \quad (41)$$

where $\mathbf{Q}(t) = [Q_1^L(t), \dots, Q_K^L(t), Q_1^E(t), \dots, Q_K^E(t)]$.

Using standard Lyapunov drift analysis, there exists a constant $B > 0$ such that

$$\Delta(t) \leq B + \mathbb{E}[\Xi(t) \mid \mathbf{Q}(t)], \quad (42)$$

where

$$\begin{aligned} \Xi(t) = & \sum_{i \in \mathcal{K}} Q_i^L(t) (A_i(t) - \mu_i^L(t) - \mu_i^T(t) - m_i^L(t)) \\ & + \sum_{i \in \mathcal{K}} Q_i^E(t) (\mu_i^T(t) - \mu_i^E(t) - m_i^E(t)). \end{aligned} \quad (43)$$

B. Drift-Plus-Penalty Minimization

Define the drift-plus-penalty as

$$\Delta_V(t) = \Delta(t) + V \mathbb{E}[C(t) \mid \mathbf{Q}(t)], \quad (44)$$

where $V > 0$ is the Lyapunov parameter. By substituting (42) into (44), we have

$$\Delta_V(t) \leq B + \mathbb{E}[\Psi(t) \mid \mathbf{Q}(t)], \quad (45)$$

where

$$\Psi(t) = \Xi(t) + VC(t). \quad (46)$$

Minimizing the upper bound yields the per-slot problem

$$(\mathbf{P2}) \quad \min_{\chi(t)} \Psi(t). \quad (47)$$

C. Problem Decomposition

Problem **(P2)** reveals an important separable structure. Specifically, local computing and transmission decisions are decoupled across users, while edge CPU allocation is coupled due to the shared resource constraint.

1) *Local CPU optimization*: For each user i , the local CPU allocation is obtained by solving

$$\begin{aligned} \min_{f_i^L(t)} \quad & VC_i^L(t) + V \phi_i^L m_i^L(t) - Q_i^L(t) (\mu_i^L(t) + m_i^L(t)) \\ \text{s.t.} \quad & (1)(3)(23)(35). \end{aligned}$$

Define

$$f_{i,\text{th}}^L = c_i Q_{i,D_i-1}^L(t) / \delta. \quad (48)$$

By considering the two regions induced by the $\max\{\cdot\}$ operator, the optimal solution can be obtained as

$$\tilde{f}_i^L(t) = \arg \min_{f \in \{f_{i,1}^L, f_{i,2}^L\}} J_i^L(f), \quad (49)$$

where

$$f_{i,1}^L = \left[\sqrt{\phi_i^L / (3\kappa c_i)} \right]_0^{\min\{f_{i,\text{max}}^L, f_{i,\text{th}}^L\}}, \quad (50)$$

$$f_{i,2}^L = \left[\sqrt{Q_i^L(t) / (3V\kappa c_i)} \right]_{f_{i,\text{th}}^L}^{f_{i,\text{max}}^L}, \quad (51)$$

and

$$J_i^L(f) = V\delta\kappa f^3 - \delta Q_i^L(t)f/c_i + (V\phi_i^L - Q_i^L(t)) \max\{0, Q_{i,D_i-1}^L(t) - \delta f/c_i\}. \quad (52)$$

Here, $[x]_a^b \triangleq \min\{\max\{x, a\}, b\}$, and only feasible candidates are retained.

2) *Transmission power optimization*: For each user i , the transmission power is obtained by solving

$$\begin{aligned} \min_{p_i(t)} \quad & VC_i^T(t) + (Q_i^E(t) - Q_i^L(t)) \mu_i^T(t) \\ \text{s.t.} \quad & (5), (6), (36). \end{aligned}$$

The optimal solution is obtained in closed form as

$$\tilde{p}_i(t) = [(Q_i^L(t) - Q_i^E(t))B_i/(V \ln 2) - N_0 B_i/H_i(t)]_0^{p_{i,\max}}, \quad (53)$$

where $[x]_a^b \triangleq \min\{\max\{x, a\}, b\}$.

3) *Edge CPU allocation*: The edge computing resource allocation is obtained by solving

$$\begin{aligned} \min_{\{f_i^E(t)\}} \quad & \sum_{i \in \mathcal{K}} VC_i^E(t) + V\phi_i^E m_i^E(t) - Q_i^E(t) (\mu_i^E(t) + m_i^E(t)) \\ \text{s.t.} \quad & (2), (4), (23), (37). \end{aligned}$$

To solve the edge CPU allocation problem, we first introduce the dual variable $\lambda \geq 0$ associated with the coupling constraint $\sum_{i \in \mathcal{K}} f_i^E(t) \leq f_{\max}^E$. The resulting Lagrangian can be written as

$$\mathcal{L}(\{f_i^E(t)\}, \lambda) = \sum_{i \in \mathcal{K}} (J_i^E(f_i^E(t)) + \lambda f_i^E(t)) - \lambda f_{\max}^E, \quad (54)$$

where

$$J_i^E(f) = V\delta\kappa f^3 - \delta Q_i^E(t)f/c_i + (V\phi_i^E - Q_i^E(t)) [Q_{i,D_i-1}^E(t) - \delta f/c_i]^+. \quad (55)$$

For a given λ , the problem is decomposed into a set of independent one-dimensional subproblems. Define

$$f_{i,\text{th}}^E = c_i Q_{i,D_i-1}^E(t)/\delta. \quad (56)$$

By considering the two regions induced by the $\max\{\cdot\}$ operator, the optimal solution of the i th subproblem is given by

$$\tilde{f}_i^E(t; \lambda) = \arg \min_{f \in \{f_{i,1}^E(\lambda), f_{i,2}^E(\lambda)\}} (J_i^E(f) + \lambda f), \quad (57)$$

where

$$f_{i,1}^E(\lambda) = \left[\sqrt{[V\delta\phi_i^E/c_i - \lambda]^+ / (3V\delta\kappa)} \right]_0^{f_{i,\text{th}}^E}, \quad (58)$$

$$f_{i,2}^E(\lambda) = \left[\sqrt{[\delta Q_i^E(t)/c_i - \lambda]^+ / (3V\delta\kappa)} \right]_{f_{i,\text{th}}^E}^{+\infty}, \quad (59)$$

Accordingly, the optimal solution is obtained as

$$\tilde{f}_i^E(t) = \tilde{f}_i^E(t; \lambda^*), \quad \forall i \in \mathcal{K}, \quad (60)$$

where the optimal dual variable λ^* is chosen such that

$$\sum_{i \in \mathcal{K}} \tilde{f}_i^E(t; \lambda^*) \leq f_{\max}^E, \quad (61)$$

Algorithm 1 Bisection Search for Optimal Dual Variable λ^*

```

1: Initialize  $\lambda_{\min} = 0$ ,  $\lambda_{\max} = \max_i \frac{\delta}{c_i} \max\{V\phi_i^E, Q_i^E(t)\}$ ,
   and tolerance  $\epsilon > 0$ .
2: repeat
3:   Set  $\lambda = (\lambda_{\min} + \lambda_{\max})/2$ .
4:   for each  $i \in \mathcal{K}$  do
5:     Compute  $\tilde{f}_i^E(t; \lambda)$  according to (57).
6:   end for
7:   if  $\sum_{i \in \mathcal{K}} \tilde{f}_i^E(t; \lambda) > f_{\max}^E$  then
8:     Update  $\lambda_{\min} \leftarrow \lambda$ .
9:   else
10:    Update  $\lambda_{\max} \leftarrow \lambda$ .
11:   end if
12: until  $|\lambda_{\max} - \lambda_{\min}| < \epsilon$ 
13: Set  $\lambda^* = \lambda$ .
14: Output  $\tilde{f}_i^E(t) = \tilde{f}_i^E(t; \lambda^*)$ ,  $\forall i \in \mathcal{K}$ .
```

Algorithm 2 Age-layered Deterministic-Delay Optimization (ADDO) Algorithm

```

1: for each slot  $t = 0, 1, 2, \dots$  do
2:   Observe system state  $\Theta(t)$ .
3:   Compute  $\lambda^*$  via Algorithm 1.
4:   for each user  $i \in \mathcal{K}$  do
5:     Compute  $\tilde{f}_i^L(t)$  according to (49), set  $f_i^L(t) = \tilde{f}_i^L(t)$ .
6:     Compute  $\tilde{p}_i(t)$  according to (53), set  $p_i(t) = \tilde{p}_i(t)$ .
7:     Compute  $\tilde{f}_i^E(t; \lambda^*)$  according to (57), set  $f_i^E(t) = \tilde{f}_i^E(t; \lambda^*)$ .
8:     Serve workload following the age-priority service discipline.
9:   end for
10:  Update queues according to (26)–(31).
11: end for
```

with equality holding when $\lambda^* > 0$.

Since $\sum_{i \in \mathcal{K}} \tilde{f}_i^E(t; \lambda)$ is monotonically non-increasing with respect to λ , the optimal dual variable λ^* can be efficiently obtained via a one-dimensional bisection search, as summarized in Algorithm 1.

The complete online policy is summarized in Algorithm 2. Its per-slot complexity is $O(K)$ for local CPU and power updates, and $O(K \log(1/\epsilon))$ for edge CPU allocation via bisection.

D. Performance Guarantee

1) Cost and Backlog Upper Bound:

Theorem 1: Assume arrival $\{A_i(t), \forall i, t\}$ lies in the interior of the capacity region of the system. Let C^* denote the minimum time-average expected cost of problem (P1) over all feasible policies. Then, under ADDO with control parameter $V > 0$, the following holds:

$$\bar{C} \leq C^* + B/V, \quad (62)$$

$$\bar{Q} \leq \min \left\{ (B + VC_{\max})/\epsilon, \sum_{i \in \mathcal{K}} D_i A_{i,\max} \right\}, \quad (63)$$

where $B, \epsilon > 0$, and

$$C_{\max} \triangleq \delta \kappa (f_{\max}^E)^3 + \delta \sum_{i \in \mathcal{K}} [\kappa (f_{i,\max}^L)^3 + p_{i,\max}] + \sum_{i \in \mathcal{K}} [(\phi_i^L + \phi_i^E) A_{i,\max}] \quad (64)$$

is a finite per-slot cost upper bound.

Proof 1: ADDO minimizes the right-hand side of the following inequality over all feasible control actions in each slot:

$$\Delta_V(t) \leq B + \mathbb{E}[VC(t) + \Xi(t) | \mathbf{Q}(t)]. \quad (65)$$

Therefore, it performs no worse than any feasible stationary randomized policy. Since the arrival lies in the interior of the capacity region of the system, there exists an optimal scheme which makes i.i.d. decision $\chi^*(t)$ over time independent of the queue backlog sizes such that

$$\mathbb{E}[A_i(t)] \leq \mathbb{E}[\mu_i^L(t) + \mu_i^T(t) + m_i^L(t) | \chi^*(t)], \quad (66)$$

$$\mathbb{E}[\mu_i^T(t) | \chi^*(t)] \leq \mathbb{E}[\mu_i^E(t) + m_i^E(t) | \chi^*(t)]. \quad (67)$$

By (43), we have

$$\mathbb{E}[\Xi(t) | \mathbf{Q}(t), \chi^*(t)] \leq 0. \quad (68)$$

Since ADDO minimizes the right-hand side of (65), we have

$$\begin{aligned} \Delta_V(t) &\leq B + \mathbb{E}[VC(t) + \Xi(t) | \mathbf{Q}(t), \chi^*(t)] \\ &= B + \mathbb{E}[VC(t) | \mathbf{Q}(t), \chi^*(t)] \\ &= B + VC^*. \end{aligned} \quad (69)$$

Taking full expectation and summing over $t = 0, 1, \dots, T-1$, we obtain

$$\mathbb{E}[L(T)] - \mathbb{E}[L(0)] + V \sum_{t=0}^{T-1} \mathbb{E}[C(t)] \leq TB + TVC^*. \quad (70)$$

Rearranging terms gives

$$\frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[C(t)] \leq C^* + \frac{B}{V} + \frac{\mathbb{E}[L(0)] - \mathbb{E}[L(T)]}{VT}. \quad (71)$$

Since $\mathbb{E}[L(T)] \geq 0$ and $\mathbb{E}[L(0)] < \infty$, taking $T \rightarrow \infty$ yields (62).

For each user i , the total workload in age layer a satisfies

$$Q_{i,a}^L(t) + Q_{i,a}^E(t) \leq A_{i,\max}, \quad a = 0, 1, \dots, D_i - 1. \quad (72)$$

Consequently, the total backlog of user i is deterministically bounded as

$$Q_i^L(t) + Q_i^E(t) \leq D_i A_{i,\max}, \quad \forall t, \quad (73)$$

Taking long-term time-averaged expectation of the left-hand side of above inequality, we have

$$\bar{Q} \leq \sum_{i \in \mathcal{K}} D_i A_{i,\max}. \quad (74)$$

Since the arrival lies in the interior of the capacity region of the system, there must exist some $\epsilon > 0$ such that exists a

feasible scheme which makes i.i.d. decision $\chi^\epsilon(t)$ over time independent of the queue backlog sizes such that

$$\mathbb{E}[A_i(t)] \leq \mathbb{E}[\mu_i^L(t) + \mu_i^T(t) + m_i^L(t) | \chi^\epsilon(t)] - \epsilon, \quad (75)$$

$$\mathbb{E}[\mu_i^T(t) | \chi^\epsilon(t)] \leq \mathbb{E}[\mu_i^E(t) + m_i^E(t) | \chi^\epsilon(t)] - \epsilon. \quad (76)$$

By (43), we have

$$\mathbb{E}[\Xi(t) | \mathbf{Q}(t), \chi^\epsilon(t)] \leq \epsilon \sum_{i \in \mathcal{K}} (Q_i^L(t) + Q_i^E(t)). \quad (77)$$

Since ADDO minimizes the right-hand side of (65), we have

$$\begin{aligned} \Delta_V(t) &\leq B + \mathbb{E}[VC(t) + \Xi(t) | \Theta(t)] \\ &\leq B + \mathbb{E}[VC(t) + \Xi(t) | \Theta(t), \chi^\epsilon(t)] \\ &\leq B + \mathbb{E}[VC(t) | \mathbf{Q}(t), \chi^\epsilon(t)] \\ &\quad - \epsilon \sum_{i \in \mathcal{K}} (Q_i^L(t) + Q_i^E(t)) \\ &\leq B + VC_{\max} - \epsilon \sum_{i \in \mathcal{K}} (Q_i^L(t) + Q_i^E(t)). \end{aligned} \quad (78)$$

Taking full expectation, we obtain

$$\begin{aligned} \mathbb{E}[L(t+1)] - \mathbb{E}[L(t)] + V \mathbb{E}[C(t)] \\ \leq B + VC_{\max} + \epsilon \sum_{i \in \mathcal{K}} \mathbb{E}[Q_i^L(t) + Q_i^E(t)]. \end{aligned} \quad (79)$$

Rearranging terms gives

$$\begin{aligned} \epsilon \sum_{i \in \mathcal{K}} \mathbb{E}[Q_i^L(t) + Q_i^E(t)] \\ \leq B + VC_{\max} - V \mathbb{E}[C(t)] + \mathbb{E}[L(t)] - \mathbb{E}[L(t+1)] \\ \leq B + VC_{\max} + \mathbb{E}[L(t)] - \mathbb{E}[L(t+1)]. \end{aligned} \quad (80)$$

Summing over $t = 0, 1, \dots, T-1$, we obtain

$$\begin{aligned} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i \in \mathcal{K}} \mathbb{E}[Q_i^L(t) + Q_i^E(t)] \\ \leq \frac{B + VC_{\max}}{\epsilon} + \frac{\mathbb{E}[L(0)] - \mathbb{E}[L(T)]}{\epsilon T}. \end{aligned} \quad (81)$$

Since $\mathbb{E}[L(T)] \geq 0$ and $\mathbb{E}[L(0)] < \infty$, taking $T \rightarrow \infty$ yields

$$\bar{Q} \leq \frac{B + VC_{\max}}{\epsilon}. \quad (82)$$

Combining with (74), (63) is proven.

2) Deadline Violations:

Theorem 2 (Average deadline-violation bound): Define the time-average expected deadline-violation amount as

$$\bar{M} \triangleq \limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \sum_{i \in \mathcal{K}} \mathbb{E}[m_i^L(t) + m_i^E(t)]. \quad (83)$$

Let

$$\phi_{\min} \triangleq \min_{i \in \mathcal{K}} \{\phi_i^L, \phi_i^E\} > 0. \quad (84)$$

Then, under ADDO,

$$\bar{M} \leq \frac{\bar{C}}{\phi_{\min}} \leq \frac{C^*}{\phi_{\min}} + \frac{B}{V \phi_{\min}}. \quad (85)$$

Proof 2: By the per-slot cost definition (34), we have

$$\begin{aligned} C(t) &\geq \sum_{i \in \mathcal{K}} (\phi_i^L m_i^L(t) + \phi_i^E m_i^E(t)) \\ &\geq \phi_{\min} \sum_{i \in \mathcal{K}} (m_i^L(t) + m_i^E(t)). \end{aligned} \quad (86)$$

Taking expectation, time average, and $\limsup$ on both sides yields

$$\bar{M} \leq \bar{C} / \phi_{\min}. \quad (87)$$

Combining this with (62) proves (85).

VI. NUMERICAL RESULTS

A. Simulation Setup

We consider an MEC system with one edge server and 20 users. Time is slotted with duration $\delta = 10$ ms, and each simulation runs for 10^4 slots. For each user i , the workload arrivals are i.i.d. with $A_i(t) \sim \text{Unif}(0, 3 \times 10^4)$ bits, and each bit requires $c_i \sim \text{Unif}(500, 1000)$ CPU cycles. The task deadline D_i is uniformly drawn from $\{D_{\min}, D_{\min}+1, \dots, D_{\max}\}$, where $D_{\min} = 2$ and $D_{\max} = 10$ unless otherwise specified.

For the wireless uplink, we set the reference channel gain $\beta_0 = 10^{-4}$, path-loss exponent $\alpha = 4$, user-edge distance 150 m, noise power spectral density $N_0 = -174$ dBm/Hz, bandwidth $B_i = 1$ MHz, and maximum transmit power $p_{i,\max} = 500$ mW. Each user has a local CPU with $f_{i,\max}^L = 1$ GHz, while the edge server provides a shared CPU capacity of $f_{\max}^E = 10$ GHz. The DVFS coefficient is $\kappa = 10^{-28}$, and the deadline-violation penalties are $\phi_i^L = \phi_i^E = 0.01$. The control parameter V is varied from 10^8 to 2×10^{11} . All simulated settings satisfy $V\phi_i^E \geq D_i A_{i,\max}$.

We compare ADDO with five baselines: *ADDO-SQ*, an ablated variant of ADDO using a conventional single queue instead of age-layered queues; the *delay-aware energy-efficient (DAEE) scheme* [18], which uses virtual queues and perturbed Lyapunov optimization to mitigate deadline violations; *LocalOnly*, where all workloads are processed locally; *EdgeOnly*, where all workloads are offloaded to the edge server; and *Random*, which uniformly allocates communication and computation resources over the feasible action space. The performance is evaluated by average energy cost, deadline-violation ratio, and average delay per bit. The source code for reproducing the numerical results is publicly available online.⁴

B. Simulation Results

TABLE I: Performance Comparison of Different Policies

Policy	Energy Cost (J/slot)	Deadline Violation Ratio	Average Delay (slots)
ADDO	0.0453	0.0001	1.5768
ADDO-SQ	0.0440	0.0194	1.5602
DAEE	0.0564	0.0320	1.5719
LocalOnly	0.0200	0.1520	4.4649
EdgeOnly	0.1052	0.5550	6.0248
Random	0.0679	0.2235	3.5854

⁴Code: <https://github.com/Sydne/addo/tree/main/code>.

1) *Comparison with baseline policies:* Table I compares ADDO with five baselines, where $V = 10^{11}$ for ADDO, ADDO-SQ, and DAEE. ADDO achieves the lowest deadline-violation ratio of 0.0001, representing reductions of 99.48% and 99.69% relative to ADDO-SQ and DAEE, respectively. Its average energy cost is 0.0453 J/slot, 19.68% lower than DAEE and only 2.95% higher than ADDO-SQ, while the three schemes exhibit comparable average delay. In contrast, although LocalOnly achieves the lowest energy cost, it suffers from a substantially higher violation ratio and delay, while EdgeOnly and Random also exhibit poor deadline reliability. These results demonstrate that the age-layered queue representation substantially improves deadline reliability with only marginal energy and delay overhead.

2) *Comparison of ADDO, ADDO-SQ, and DAEE under different V :* Fig. 2 compares ADDO, ADDO-SQ, and DAEE as V varies from 10^8 to 2×10^{11} . ADDO consistently outperforms ADDO-SQ and DAEE in deadline reliability: its violation ratio remains near-zero for $V \leq 1.5 \times 10^{11}$ and is 96.31% and 96.20% lower (0.0024 vs. 0.0650 and 0.0631) even at $V = 2 \times 10^{11}$. This comparison indicates that the age-layered queue representation, rather than merely the Lyapunov-based resource controller, is responsible for the main reliability improvement.

The performance degradation at large V aligns with Theorem 1, which implies an $[O(1/V), O(V)]$ cost-backlog trade-off; higher V prioritizes energy saving, potentially increasing workload age. Furthermore, since Theorem 2 provides an upper bound on average violations rather than guaranteeing monotonic empirical improvement, a moderate V is preferable to balance energy, delay, and reliability.

3) *Performance versus the maximum deadline $D_{\max}$:* Fig. 3 shows the performance of different policies as $D_{\max}$ varies from 2 to 10 slots, with $V = 10^{11}$ for ADDO, ADDO-SQ, and DAEE. As shown in Fig. 3a, the violation ratio decreases as the deadline range is relaxed. ADDO consistently provides the best deadline reliability. Its violation ratio decreases from 0.0131 at $D_{\max} = 2$ to nearly zero for $D_{\max} \geq 3$. Notably, even the worst-case violation ratio of ADDO (0.0131) is lower than the best value achieved by the competing policies (0.0194 for ADDO-SQ), demonstrating the robustness of ADDO under stringent deadline settings.

Fig. 3b shows that the average delays of ADDO, ADDO-SQ, and DAEE remain relatively stable as $D_{\max}$ increases, whereas those of LocalOnly, EdgeOnly, and Random increase substantially. In particular, when $D_{\max}$ increases from 2 to 10, the average delay of ADDO increases by only 13.41%, from 1.3904 to 1.5768 slots, while that of Random increases by 116.17%, from 1.6586 to 3.5854 slots. This result indicates that ADDO effectively suppresses queue buildup across different deadline settings.

As shown in Fig. 3c, the energy cost of ADDO decreases from 0.0532 J/slot at $D_{\max} = 2$ to 0.0453 J/slot for $D_{\max} = 10$, yielding a reduction of approximately 14.85%. ADDO also consumes less energy than DAEE over the entire range of $D_{\max}$. These results demonstrate that ADDO remains robust to

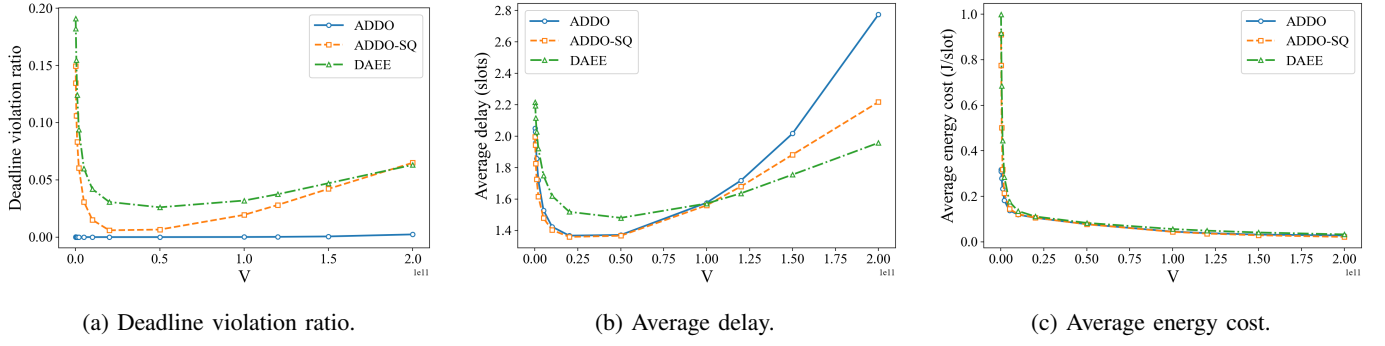


Fig. 2: Performance versus the control parameter V under ADDO, ADDO-SQ, and DAEE.

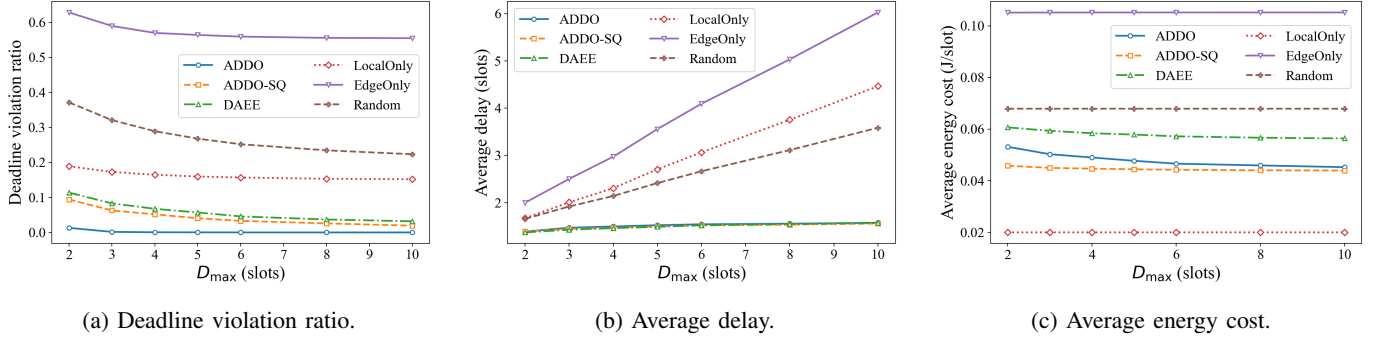


Fig. 3: Performance versus the deadline $D_{\max}$ under different policies.

variations in the deadline requirement, maintaining near-zero violations and stable delay while reducing energy consumption as the deadline is relaxed.

VII. CONCLUSION

This paper investigated online resource allocation for MEC networks under hard end-to-end delay constraints. To explicitly characterize workload urgency, we developed an age-layered queueing model for both local and edge queues, together with an age-priority service discipline for deadline-aware processing. Based on this model, we formulated a long-term stochastic optimization problem that jointly considers local computing energy, transmission energy, edge computing energy, and deadline-violation penalties. We further proposed the ADDO algorithm based on Lyapunov drift-plus-penalty optimization, which makes online decisions on local computing, task offloading, and edge CPU allocation without requiring prior knowledge of traffic-arrival or channel-state statistics. Theoretical analysis established an $O(1/V)$ cost-optimality gap. Numerical results demonstrated that ADDO achieves lower deadline-violation ratios and a more favorable delay–energy–reliability tradeoff than benchmark policies, and can eliminate deadline violations under proper parameter settings. Future work will investigate cooperative multi-edge systems and non-i.i.d., bursty, and temporally correlated traffic.

REFERENCES

- [1] C.-X. Wang, X. You, X. Gao, X. Zhu, Z. Li, C. Zhang, H. Wang, Y. Huang, Y. Chen, H. Haas *et al.*, “On the road to 6G: Visions, requirements, key technologies, and testbeds,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 905–974, 2023.
- [2] M. Ergen, B. Saoud, I. Shayea, A. A. El-Saleh, O. Ergen, F. Inan, and M. F. Tuysuz, “Edge computing in future wireless networks: A comprehensive evaluation and vision for 6G and beyond,” *ICT Express*, vol. 10, no. 5, pp. 1151–1173, 2024.
- [3] J. Sasiain, D. Franco, A. Atutxa, J. Astorga, and E. Jacob, “Toward the integration and convergence between 5G and TSN technologies and architectures for industrial communications: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 27, no. 1, pp. 259–321, 2025.
- [4] J. Ren, G. Yu, Y. Cai, and Y. He, “Latency optimization for resource allocation in mobile-edge computation offloading,” *IEEE Transactions on Wireless Communications*, vol. 17, no. 8, pp. 5506–5519, 2018.
- [5] Q. Wang, X. Chen, and Q. Qi, “Design of delay-optimal robust edge computing in 6G wireless networks,” in *IEEE WCSP*, 2022, pp. 226–231.
- [6] Z. Nan, Y. Han, S. Zhou, and Z. Niu, “Robust DNN partitioning and resource allocation under uncertain inference time,” *IEEE Transactions on Mobile Computing*, vol. 25, no. 3, pp. 3680–3696, 2026.
- [7] M. Neely, *Stochastic network optimization with application to communication and queueing systems*. Morgan & Claypool Publishers, 2010.
- [8] Y. Mao, J. Zhang, and K. B. Letaief, “Dynamic computation offloading for mobile-edge computing with energy harvesting devices,” *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3590–3605, 2016.
- [9] H. Chahed and A. Kassler, “TSN network scheduling—challenges and approaches,” *Network*, vol. 3, no. 4, pp. 585–624, 2023.
- [10] A. Nasrallah, A. S. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. ElBakoury, “Ultra-low latency (ULL) networks: The IEEE TSN and IETF DetNet standards and related 5G ULL research,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 88–145, 2018.
- [11] B. S. Khan, S. Jangsher, A. Ahmed, and A. Al-Dweik, “URLLC and eMBB in 5G industrial IoT: A survey,” *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1134–1163, 2022.
- [12] C. Xu, Y. Liu, C. Wang, and A. Nallanathan, “Analysis of grant-free access for URLLC service: A stochastic network calculus perspective,”

IEEE Transactions on Vehicular Technology, vol. 74, no. 9, pp. 14 949–14 954, 2025.

- [13] C.-F. Liu, M. Bennis, M. Debbah, and H. V. Poor, “Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing,” *IEEE Transactions on Communications*, vol. 67, no. 6, pp. 4132–4150, 2019.
- [14] Y. Chen, K. Li, Y. Wu, J. Huang, and L. Zhao, “Energy efficient task offloading and resource allocation in air-ground integrated MEC systems: A distributed online approach,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 8, pp. 8129–8142, 2024.
- [15] S. Xia, Z. Yao, Y. Li, Z. Xing, and S. Mao, “Distributed computing and networking coordination for task offloading under uncertainties,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5280–5294, 2024.
- [16] L. Tang, M. Xu, C. Xu, and K. Ye, “Collaborative resource management and workloads scheduling in cloud-assisted mobile edge computing across timescales,” in *IEEE ICWS*, 2024, pp. 1415–1425.
- [17] S. Yao, M. Wang, J. Ren, T. Xia, W. Wang, K. Xu, M. Xu, and H. Zhang, “Multi-agent reinforcement learning for task offloading in crowd-edge computing,” *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9289–9302, 2025.
- [18] H. Wu, J. Chen, T. N. Nguyen, and H. Tang, “Lyapunov-guided delay-aware energy efficient offloading in IIoT-MEC systems,” *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 2117–2128, 2023.